

Spring Testing with JUnit and Mockito

1. Why Testing Exists

Consider the following method:

```
public int calculateFinalPrice(  
    int price,  
    int discountPercentage) {  
  
    return price -  
        (price * discountPercentage / 100);  
}
```

The method appears to work, but writing code and proving that it behaves correctly are two different activities.

We can manually test it with:

```
Price = 1000  
Discount = 20  
Expected result = 800
```

Even if this input returns 800, several questions remain:

- What happens when the discount is 0?
- What happens when the discount is 100?
- What happens when the discount is negative?
- What happens when the discount is greater than 100?
- What happens if someone changes the formula later?

Testing verifies that the behaviour of the code matches its requirements.

Manual Testing vs Automated Testing

Manual Testing

A developer manually:

1. Starts the application.
2. Prepares a request.
3. Calls the endpoint.
4. Inspects the response.
5. Decides whether the result is correct.

This can work for a small application, but it becomes slow and unreliable as the application grows.

Automated Testing

An automated test:

1. Provides a known input.
2. Executes the production code.
3. Compares the actual result with the expected result.
4. Reports whether the test passed or failed.

```

Known input
  ↓
Execute production code
  ↓
Actual output
  ↓
Compare with expected output
  ↓
Pass or fail
  
```

What Is a Test Case?

A test case contains three fundamental parts:

Initial condition
Action
Expected result

Example:

Initial condition:
Price is ₹1000 and discount is 20%

Action:
Calculate the final price

Expected result:
Final price should be ₹800

In testing terminology, this is commonly called **Arrange, Act and Assert**.

Arrange

Prepare the data and environment required by the test.

```
int price = 1000;  
int discount = 20;
```

Act

Execute the behaviour being tested.

```
int actualPrice =  
    calculator.calculateFinalPrice(price, discount);
```

Assert

Compare the actual result with the expected result.

```
assertEquals(800, actualPrice);
```

The complete test looks like this:

```
@Test
void shouldApplyDiscountToPrice() {

    // Arrange
    int price = 1000;
    int discount = 20;

    // Act
    int actualPrice =
        calculator.calculateFinalPrice(
            price,
            discount
        );

    // Assert
    assertEquals(800, actualPrice);
}
```

Tests Verify Behaviour

Assume the requirement is:

| A discount must be between 0 and 100.

This requirement introduces multiple behaviours:

- A valid discount calculates the final price.
- A discount below 0 is rejected.
- A discount above 100 is rejected.
- A negative price is rejected.

The production code can enforce these rules:

```
public int calculateFinalPrice(
    int price,
```

```

    int discountPercentage) {

    if (price < 0) {
        throw new IllegalArgumentException(
            "Price cannot be negative"
        );
    }

    if (discountPercentage < 0 ||
        discountPercentage > 100) {

        throw new IllegalArgumentException(
            "Discount must be between 0 and 100"
        );
    }

    return price -
        (price * discountPercentage / 100);
}

```

Tests should be written for the behaviours promised by a method, not simply for its lines of code.

Unit Tests and Integration Tests

Consider a common Spring Boot application flow:

```
Controller → Service → Repository → Database
```

Unit Test

A unit test verifies a small unit of behaviour in isolation.

For example, a test for `ProductService` should not require:

- The Spring `ApplicationContext`
- Tomcat

- A real database
- An HTTP request

If `ProductService` depends on `ProductRepository`, the real repository can be replaced with a controlled test double.

Integration Test

An integration test checks whether multiple real components work together.

Examples:

```
Repository + Hibernate + H2/MySQL
```

```
Controller + Service + Repository + Database
```

The important question is not only which annotation is used. The real question is:

How much real application infrastructure participates in the test?

Test type	What is real?	Relative speed	Main purpose
Unit test	One class or unit of behaviour	Fastest	Verify business logic
Slice test	One application layer	Fast	Verify a specific Spring layer
Integration test	Multiple components	Slower	Verify that components work together
End-to-end test	Complete application flow	Slowest	Verify the complete user journey

Testing Pyramid

A healthy test suite generally contains:

```
Many fast unit tests  
Fewer integration tests  
Very few end-to-end tests
```

If every test starts Spring Boot and connects to a database, the test suite becomes slow. Developers are then less likely to run it frequently.

Unit tests provide fast feedback. Integration tests confirm that individual units are connected correctly. Both are necessary because they solve different problems.

What Should Be Unit Tested?

Prioritize code that contains behaviour or decisions:

- Business calculations
- Conditional branches
- Data transformations
- Validation rules
- Exception conditions
- Authorization decisions
- Service-layer orchestration
- Important edge cases

Usually avoid spending time unit-testing:

- Plain getters and setters
- Framework behaviour
- Built-in methods of `JpaRepository`
- Code with no meaningful decision or transformation

There is no need to prove that Spring Data's `save()` method works. The important part is whether the application calls `save()` under the correct business conditions.

2. JUnit Fundamentals

What Is JUnit?

JUnit provides:

- A programming model for declaring tests
- Annotations such as `@Test`
- Assertions such as `assertEquals()`
- Test discovery and execution
- Integration with Maven, Gradle and IDEs

The annotations used in modern Spring Boot tests belong to JUnit Jupiter:

```
org.junit.jupiter.api.Test
```

Testing Dependency

A Spring Boot project normally includes:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

The `test` scope means that these libraries are available for compiling and running tests, but they are not included as normal production dependencies.

`spring-boot-starter-test` provides commonly used testing libraries and utilities, including:

- JUnit
- Mockito
- AssertJ
- Hamcrest
- Spring Test
- JSON testing utilities

Test Directory Structure

Production code belongs in:

```
src/main/java
```

Test code belongs in:

```
src/test/java
```

The test package should normally mirror the production package:

```
src/main/java
└─ com.example.demo.service
   └─ ProductService.java

src/test/java
└─ com.example.demo.service
   └─ ProductServiceTest.java
```

This keeps the production class and its corresponding test easy to locate.

Writing the First JUnit Test

Production Class

```
public class PriceCalculator {

    public int calculateFinalPrice(
        int price,
        int discountPercentage) {

        if (price < 0) {
            throw new IllegalArgumentException(
                "Price cannot be negative"
            );
        }
    }
}
```

```

    }

    if (discountPercentage < 0 ||
        discountPercentage > 100) {

        throw new IllegalArgumentException(
            "Discount must be between 0 and 100"
        );
    }

    return price -
        (price * discountPercentage / 100);
}
}

```

Test Class

```

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class PriceCalculatorTest {

    private PriceCalculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new PriceCalculator();
    }

    @Test
    void shouldApplyDiscountToPrice() {

        // Arrange
    }
}

```

```
int price = 1000;
int discount = 20;

// Act
int actualPrice =
    calculator.calculateFinalPrice(
        price,
        discount
    );

// Assert
assertEquals(800, actualPrice);
}
```

This test does not use:

```
@SpringBootTest
@Autowired
```

It also does not start the Spring `ApplicationContext`. `PriceCalculator` is a regular Java class, so it can be created with `new`.

Spring Boot applications are still Java applications. The complete Spring container should not be started when the behaviour can be tested directly.

Understanding `@Test`

```
@Test
void shouldApplyDiscountToPrice() {
}
```

`@Test` tells JUnit to discover and execute the method as a test case.

Test method names should describe the expected behaviour.

Good names:

```
shouldApplyDiscountToPrice()
shouldThrowExceptionWhenDiscountIsNegative()
shouldReturnOriginalPriceWhenDiscountIsZero()
```

Avoid vague names:

```
test1()
checkMethod()
calculatorTest()
```

Assertions

An assertion describes a condition that must be true for the test to pass.

assertEquals()

```
assertEquals(expectedValue, actualValue);
```

```
assertEquals(800, actualPrice);
```

assertTrue() and **assertFalse()**

```
assertTrue(result.isActive());
assertFalse(result.isDeleted());
```

assertNull() and **assertNotNull()**

```
assertNull(product.getId());
assertNotNull(savedProduct.getId());
```

assertThrows()

Use **assertThrows()** when an exception is the expected behaviour.

```
@Test
void shouldRejectDiscountAboveHundred() {

    IllegalArgumentException exception =
        assertThrows(
            IllegalArgumentException.class,
            () -> calculator.calculateFinalPrice(
                1000,
                120
            )
        );

    assertEquals(
        "Discount must be between 0 and 100",
        exception.getMessage()
    );
}
```

The test passes only when the specified code throws `IllegalArgumentException`. If no exception is thrown, the test fails.

`@BeforeEach` and `@AfterEach`

`@BeforeEach`

`@BeforeEach` runs before every test method.

```
@BeforeEach
void setUp() {
    calculator = new PriceCalculator();
}
```

It is useful for preparation shared by multiple tests.

```
setUp()
testOne()
```

```
setUp()
testTwo()

setUp()
testThree()
```

Every test begins with a predictable state.

@AfterEach

@AfterEach runs after every test method.

```
@AfterEach
void cleanup() {
    // Release test resources if required
}
```

It is useful when a test manually opens a resource that must be closed or cleaned. There is no need to add an empty @AfterEach merely because the annotation exists.

Running Tests

Tests can be run directly from IntelliJ.

To run the complete Maven test suite:

```
./mvnw test
```

If Maven is installed globally:

```
mvn test
```

```
Green → Expected behaviour was observed
Red   → Expected and actual behaviour did not match
```

A failing test provides useful information. It may mean that:

- The production code is incorrect.
- The test's expectation is incorrect.
- The requirement has changed.

3. Unit Testing the Service Layer with Mockito

The Dependency Problem

Consider the following service:

```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(
        ProductRepository productRepository) {

        this.productRepository = productRepository;
    }

    public Product getProductById(Long id) {

        return productRepository.findById(id)
            .orElseThrow(() ->
                new ProductNotFoundException(
                    "Product not found: " + id
                )
            );
    }

    public Product createProduct(Product product) {
```

```

        boolean alreadyExists =
            productRepository.existsByName(
                product.getName()
            );

        if (alreadyExists) {
            throw new IllegalArgumentException(
                "Product already exists"
            );
        }

        return productRepository.save(product);
    }
}

```

Repository:

```

public interface ProductRepository
    extends JpaRepository<Product, Long> {

    boolean existsByName(String name);
}

```

`ProductService` cannot be created without a `ProductRepository`:

```

ProductService service =
    new ProductService(???);

```

Using the real repository could also require:

- A Spring Data proxy
- `EntityManager`
- Hibernate
- `DataSource`

- A database

A failure could then come from the database or Hibernate configuration instead of the service logic. That would no longer be an isolated unit test.

The dependency should therefore be replaced with a controlled test double.

Test Doubles

A test double is an object used in place of a real dependency during testing.

Dummy

An object passed only because a method requires it. Its behaviour is not used by the test.

Stub

An object that returns predefined answers.

```
When findById(1) is called,  
return this predefined product.
```

Fake

A simplified but working implementation, such as an in-memory repository backed by a `HashMap` instead of MySQL.

Mock

A configurable test double that can:

- Return predefined values
- Throw predefined exceptions
- Record method calls
- Verify how it was used

Mockito creates mocks without requiring manually written fake repository classes.

Configuring Mockito with JUnit

```
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock
    private ProductRepository productRepository;

    @InjectMocks
    private ProductService productService;
}
```

@ExtendWith(MockitoExtension.class)

JUnit knows how to execute tests, but it does not process Mockito annotations by itself.

`MockitoExtension` connects Mockito's lifecycle with JUnit Jupiter and initializes fields annotated with:

```
@Mock
@InjectMocks
```

@Mock

```
@Mock
private ProductRepository productRepository;
```

Mockito creates an object that implements `ProductRepository`. This object is not:

- A Spring bean
- A Spring Data proxy

- A real repository
- Connected to a database

It is a controlled replacement used only by the test.

@InjectMocks

```
@InjectMocks
private ProductService productService;
```

Mockito creates `ProductService` and injects the available mock into its constructor.

Conceptually, it performs something similar to:

```
ProductRepository repository =
    Mockito.mock(ProductRepository.class);

ProductService productService =
    new ProductService(repository);
```

`@InjectMocks` is a Mockito convenience mechanism. It is not Spring dependency injection.

Testing the Success Scenario

```
@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock
    private ProductRepository productRepository;

    @InjectMocks
    private ProductService productService;

    @Test
    void shouldReturnProductWhenProductExists() {
```

```
// Arrange
Product product =
    new Product(
        1L,
        "Laptop",
        10
    );

when(productRepository.findById(1L))
    .thenReturn(Optional.of(product));

// Act
Product result =
    productService.getProductById(1L);

// Assert
assertEquals(1L, result.getId());
assertEquals("Laptop", result.getName());

verify(productRepository)
    .findById(1L);
}
}
```

The following stubbing instruction controls the mock repository:

```
when(productRepository.findById(1L))
    .thenReturn(Optional.of(product));
```

It means:

If findById(1L) is called on this mock repository,
return the predefined Product.

Mockito is not testing the repository. It is creating a predictable environment in which the service can be tested.

Stubbing, Assertion and Verification

These three concepts have different responsibilities.

Stubbing

```
when(repository.findById(1L))  
    .thenReturn(Optional.of(product));
```

Defines how the dependency should behave during the test.

Assertion

```
assertEquals("Laptop", result.getName());
```

Checks the observable result produced by the service.

Verification

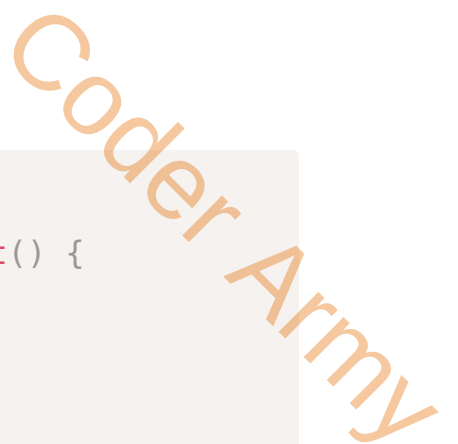
```
verify(repository).findById(1L);
```

Checks whether the service interacted with its dependency as expected.

```
when()    → Control the dependency  
assert() → Check the result  
verify() → Check the interaction
```

`verify()` does not need to be called for every mocked method. Use it when the interaction itself is part of the behaviour being tested.

Testing the Exception Scenario



```

@Test
void shouldThrowExceptionWhenProductDoesNotExist() {

    // Arrange
    when(productRepository.findById(99L))
        .thenReturn(Optional.empty());

    // Act
    ProductNotFoundException exception =
        assertThrows(
            ProductNotFoundException.class,
            () -> productService
                .getProductById(99L)
        );

    // Assert
    assertEquals(
        "Product not found: 99",
        exception.getMessage()
    );

    verify(productRepository)
        .findById(99L);
}

```

The repository is instructed to report that the product is absent. The test then verifies that the service throws `ProductNotFoundException`.

This checks the application's business rule, not the database.

Testing Product Creation

Successful Creation

```
@Test
void shouldCreateProductWhenNameIsUnique() {

    // Arrange
    Product request =
        new Product(
            null,
            "Keyboard",
            20
        );

    Product savedProduct =
        new Product(
            10L,
            "Keyboard",
            20
        );

    when(productRepository.existsByName("Keyboard"))
        .thenReturn(false);

    when(productRepository.save(request))
        .thenReturn(savedProduct);

    // Act
    Product result =
        productService.createProduct(request);

    // Assert
    assertEquals(10L, result.getId());
    assertEquals("Keyboard", result.getName());

    verify(productRepository)
        .existsByName("Keyboard");
}
```

```
        verify(productRepository)
            .save(request);
    }
}
```

Duplicate Product

```
@Test
void shouldRejectProductWhenNameAlreadyExists() {

    // Arrange
    Product request =
        new Product(
            null,
            "Keyboard",
            20
        );

    when(productRepository.existsByName("Keyboard"))
        .thenReturn(true);

    // Act and Assert
    assertThrows(
        IllegalArgumentException.class,
        () -> productService.createProduct(request)
    );

    verify(productRepository)
        .existsByName("Keyboard");

    verify(productRepository, never())
        .save(any(Product.class));
}
```

The following verification is important:

```
verify(productRepository, never())  
    .save(any(Product.class));
```

The business rule states that a duplicate product must not be saved. The absence of the `save()` call is therefore part of the required behaviour.

Argument Matchers

Exact-Value Stubbing

```
when(productRepository.findById(1L))  
    .thenReturn(Optional.of(product));
```

Flexible Stubbing

```
when(productRepository.save(any(Product.class)))  
    .thenReturn(savedProduct);
```

Common Mockito matchers include:

```
any()  
anyLong()  
anyString()  
eq(value)
```

Example:

```
verify(productRepository)  
    .save(any(Product.class));
```

Use exact values when the exact argument matters. Use matchers when the test only cares about the argument's type or category.

When a matcher is used for one argument of a multi-argument method, use matchers for all arguments:

```
when(repository.search(
    eq("Laptop"),
    eq(true)
)).thenReturn(products);
```

4. Testing Spring MVC Controllers

What Should a Controller Test Verify?

Consider the following controller:

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(
        ProductService productService) {

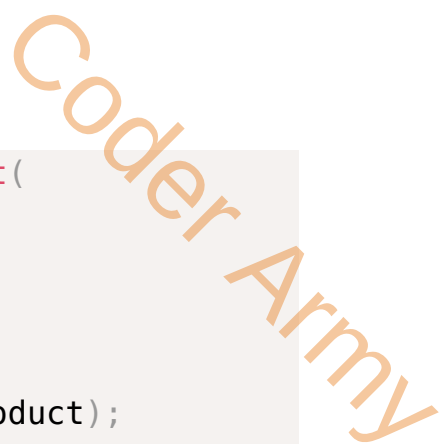
        this.productService = productService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProduct(
        @PathVariable Long id) {

        Product product =
            productService.getProductById(id);

        return ResponseEntity.ok(product);
    }

    @PostMapping
```



```

public ResponseEntity<Product> createProduct(
    @Valid
    @RequestBody Product product) {

    Product savedProduct =
        productService.createProduct(product);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(savedProduct);
}
}

```

The controller is responsible for the HTTP contract:

- Which URL is accepted?
- Which HTTP method is accepted?
- How is JSON converted into Java?
- Are path variables and request parameters read correctly?
- Is validation executed?
- Which status code is returned?
- Which JSON is returned?
- How are exceptions converted into HTTP responses?
- Does security permit or reject the request?

A controller test should not repeat all service business-logic tests. That behaviour belongs in the service unit tests.

Why Not Call the Controller Method Directly?

It is possible to write:

```

ProductController controller =
    new ProductController(mockService);

```

```
ResponseEntity<Product> response =  
    controller.getProduct(1L);
```

This tests the Java method, but it does not verify Spring MVC behaviour such as:

- `@GetMapping("/{id}")`
- `@PathVariable` conversion
- `@RequestBody` deserialization
- `@Valid`
- HTTP status handling
- JSON serialization
- Spring Security filters
- Global exception handlers

The controller should therefore be tested through Spring MVC's request-processing infrastructure.

`@WebMvcTest`

```
@WebMvcTest(ProductController.class)  
class ProductControllerTest {  
}
```

`@WebMvcTest` creates a limited Spring `ApplicationContext` containing components relevant to the web layer.

Loaded:

```
Controller  
DispatcherServlet  
Jackson JSON conversion  
Validation  
Controller advice  
MVC configuration  
Security filters
```

Not loaded:

Real service
Real repository
Hibernate
Database
Complete application

This is known as a **test slice**: only the part of the application required by the test is loaded.

MockMvc

```
@Autowired  
private MockMvc mockMvc;
```

MockMvc creates an HTTP-like request without starting a real web server.

```
Mock HTTP request  
    ↓  
Spring Security filter chain  
    ↓  
DispatcherServlet  
    ↓  
Controller  
    ↓  
HTTP-like response
```

There is no:

localhost
Port 8080
Running Tomcat server
Postman

Spring MVC still processes the request using its real web infrastructure.

@Mock vs @MockitoBean

The service unit test used:

```
@Mock
private ProductRepository productRepository;
```

That test did not have a Spring context, so Mockito created a normal mock field.

In a controller test, Spring creates `ProductController`, whose constructor requires a `ProductService` bean. The mock must therefore be placed inside Spring's test context:

```
@MockitoBean
private ProductService productService;
```

`@MockitoBean` creates or replaces the corresponding bean in the Spring test `ApplicationContext` with a Mockito mock.

Annotation	Managed by	Placed in the Spring context?
<code>@Mock</code>	Mockito	No
<code>@MockitoBean</code>	Spring Test and Mockito	Yes

For current Spring versions, use `@MockitoBean`. Older Spring Boot projects may use the now-deprecated `@MockBean`.

Testing a GET Endpoint

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private ProductService productService;

    @Test
    @WithMockUser
    void shouldReturnProductWhenProductExists()
```

```

        throws Exception {

// Arrange
Product product =
    new Product(
        1L,
        "Laptop",
        10
    );

when(productService.getProductById(1L))
    .thenReturn(product);

// Act and Assert
mockMvc.perform(
    get("/api/products/1")
        .accept(
            MediaType.APPLICATION
_JSON
        )
    )
    .andExpect(
        status().isOk()
    )
    .andExpect(
        content().contentTypeCompatibleWith(
            MediaType.APPLICATION_JSON
        )
    )
    .andExpect(
        jsonPath("$.id").value(1)
    )
    .andExpect(
        jsonPath("$.name").value("Laptop")
    )
    .andExpect(

```

```

        jsonPath("$.stock").value(10)
    );

    verify(productService)
        .getProductById(1L);
}
}

```

The test sends:

```
GET /api/products/1
```

Spring MVC must:

1. Match the request with `@GetMapping("/{id}")`.
2. Extract `1` from the URL.
3. Convert it into a `Long`.
4. Call the controller method.
5. Serialize the returned `Product` into JSON.
6. Generate an HTTP response.

perform()

```

mockMvc.perform(
    get("/api/products/1")
);

```

Creates and executes the mock request.

status()

```
.andExpect(status().isOk());
```

Verifies that the response status is `200 OK`.

content()

```
.andExpect(
    content().contentTypeCompatibleWith(
        MediaType.APPLICATION_JSON
    )
);
```

Verifies that the response content represents JSON.

jsonPath()

```
.andExpect(
    jsonPath("$.name").value("Laptop")
);
```

For the following response:

```
{
  "id": 1,
  "name": "Laptop",
  "stock": 10
}
```

The JSON paths are:

```
$.id
$.name
$.stock
```

Testing a POST Endpoint

```
@Test
@WithMockUser
void shouldCreateProductWhenRequestIsValid()
```

```

        throws Exception {

// Arrange
Product savedProduct =
    new Product(
        10L,
        "Keyboard",
        20
    );

when(productService.createProduct(
    any(Product.class)
)).thenReturn(savedProduct);

String requestBody = """
    {
        "name": "Keyboard",
        "stock": 20
    }
    """;

// Act and Assert
mockMvc.perform(
    post("/api/products")
        .with(csrf())
        .contentType(
            MediaType.APPLICATION_JSON
N
        )
        .content(requestBody)
    )
    .andExpect(
        status().isCreated()
    )
    .andExpect(
        jsonPath("$.id").value(10)
    )

```

```

        )
        .andExpect(
            jsonPath("$.name").value("Keyboard")
        )
        .andExpect(
            jsonPath("$.stock").value(20)
        );

        verify(productService)
            .createProduct(any(Product.class));
    }

```

This test verifies that:

- The `POST` mapping works.
- JSON is converted into a `Product` object.
- The controller calls the service.
- The response status is `201 Created`.
- The saved product is converted into response JSON.

For unsafe HTTP methods such as `POST`, `PUT` and `DELETE`, include:

```

.with(csrf())

```

when CSRF protection is enabled.

Handling Spring Security

If Spring Security protects the endpoint, an unauthenticated request may return:

```

401 Unauthorized

```

Use `@WithMockUser` to execute a test as an authenticated user:

```
@Test
@WithMockUser(
    username = "aditya",
    roles = "ADMIN"
)
void shouldAllowAdminToCreateProduct() {
}
```

For Spring Boot 4, add the technology-specific test starter:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security-test</artifactId>
</dependency>
<scope>test</scope>
```

5. Repository and Integration Testing

Why Repository Tests Use a Database

Mocking a repository does not prove that the real repository query works:

```
ProductRepository repository =
    mock(ProductRepository.class);

when(repository.findByNameIgnoreCase("laptop"))
    .thenReturn(Optional.of(product));
```

This only proves that Mockito returns the value configured in the test. It does not execute or validate the real query.

@DataJpaTest

```
@DataJpaTest
class ProductRepositoryTest {
}
```

`@DataJpaTest` loads the JPA-related slice of the application.

Loaded:

Entities
Spring Data repositories
Hibernate
EntityManager
DataSource
Embedded database

Not loaded:

Controllers
Normal service beans
Web server
Complete application

When an embedded database is present, Spring Boot configures it for the test. Data JPA tests are transactional and their changes are rolled back after each test.

Adding H2 for Tests

H2 is an embedded database that runs inside the test process.

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>
```

Benefits of H2 for tests:

- No separate installation
- No database server to start

- Fast test execution
- A fresh and isolated test environment

Quick Comparison

Area under test	Main tool or annotation	Real components	Replaced components	Main confidence provided
Plain Java logic	JUnit and <code>@Test</code>	Class under test	None	Calculations, validation and edge cases behave correctly
Service layer	Mockito, <code>@Mock</code> , <code>@InjectMocks</code>	Service	Repository or other dependencies	Business logic and service orchestration behave correctly
MVC controller	<code>@WebMvcTest</code> , <code>MockMvc</code> , <code>@MockitoBean</code>	Spring MVC web layer	Service	HTTP mappings, JSON, status codes, validation and security behave correctly
Repository	<code>@DataJpaTest</code> and H2	Repository, Hibernate and database	Other application layers	Entity mappings and repository queries work correctly

Core Principle

Use the smallest test environment that can prove the behaviour being tested:

```
Plain Java behaviour → JUnit
Service with dependencies → JUnit + Mockito
Web-layer contract → @WebMvcTest + MockMvc
Repository query or mapping → @DataJpaTest + Database
```

Fast unit tests and focused integration tests work together to provide reliable feedback without making the test suite unnecessarily slow.

Coder Army